UNIT3: Object Oriented Programming & Files

Python OOPs Concepts

Python is an object-oriented language since its beginning. It allows us to develop applications using an Object-Oriented approach. In Python, we can easily create and use classes and objects.

An object-oriented paradigm is to design the program using classes and objects. The object is related to real-word entities such as book, house, pencil, etc. The oops concept focuses on writing the reusable code. It is a widespread technique to solve the problem by creating objects.

Major principles of object-oriented programming system are given below.

- Class
- Object
- Method
- Inheritance
- Polymorphism
- Data Abstraction
- Encapsulation

Class

The class can be defined as a collection of objects. It is a logical entity that has some specific attributes and methods. For example: if you have an employee class, then it should contain an attribute and method, i.e. an email id, name, age, salary, etc.

Syntax

```
class ClassName:
       <statement-1>
        .
        .
```

```
        <statement-N>
```

Object

The object is an entity that has state and behavior. It may be any real-world object like the mouse, keyboard, chair, table, pen, etc.

Everything in Python is an object, and almost everything has attributes and methods. All functions have a built-in attribute __doc__, which returns the docstring defined in the function source code.

When we define a class, it needs to create an object to allocate the memory. Consider the following example.

Example:

```
class car:
    def __init__(self,modelname, year):
        self.modelname = modelname
        self.year = year
    def display(self):
        print(self.modelname,self.year)

c1 = car("Toyota", 2016)
c1.display()
```

Output:

```
Toyota 2016
```

In the above example, we have created the class named car, and it has two attributes modelname and year. We have created a c1 object to access the class attribute. The c1 object will allocate memory for these values. We will learn more about class and object in the next tutorial.

Method

In Python, methods are not restricted to user-defined class instances; since everything is an object, built-in data ty and integers also have their own methods.

The method is a function that is associated with an object. In Python, a method is not unique to class instances. Any object type can have methods.

Inheritance

Inheritance is the most important aspect of object-oriented programming, which simulates the real-world concept of inheritance. It specifies that the child object acquires all the properties and behaviors of the parent object.

By using inheritance, we can create a class which uses all the properties and behavior of another class. The new class is known as a derived class or child class, and the one whose properties are acquired is known as a base class or parent class.

It provides the re-usability of the code.

Polymorphism

Polymorphism contains two words "poly" and "morphs". Poly means many, and morph means shape. By polymorphism, we understand that one task can be performed in different ways. For example - you have a class animal, and all animals speak. But they speak differently. Here, the "speak" behavior is polymorphic in a sense and depends on the animal. So, the abstract "animal" concept does not actually "speak", but specific animals (like dogs and cats) have a concrete implementation of the action "speak".

Encapsulation

Encapsulation is also an essential aspect of object-oriented programming. It is used to restrict access to methods and variables. In encapsulation, code and data are wrapped together within a single unit from being modified by accident.

Data Abstraction

Data abstraction and encapsulation both are often used as synonyms. Both are nearly synonyms because data abstraction is achieved through encapsulation.
Abstraction is used to hide internal details and show only functionalities. Abstracting something means to give names to things so that the name captures the core of what a function or a whole program does.

Object-oriented vs. Procedure-oriented Programming languages

The difference between object-oriented and procedure-oriented programming is given below:

Index	Object-oriented Programming	Procedural Programming
1.	Object-oriented programming is the problem-solving approach and used where computation is done by using objects.	Procedural programming uses a list of instructions to do computation step by step.
2.	It makes the development and maintenance easier.	In procedural programming, It is not easy to maintain the codes when the project becomes lengthy.
3.	It simulates the real world entity. So real-world problems can be easily solved through oops.	It doesn't simulate the real world. It works on step by step instructions divided into small parts called functions.
4.	It provides data hiding. So it is more secure than procedural languages. You cannot access private data from anywhere.	Procedural language doesn't provide any proper way for data binding, so it is less secure.
5.	Example of object-oriented programming languages is C++, Java, .Net, Python, C#, etc.	Example of procedural languages are: C, Fortran, Pascal, VB etc.

Python Class and Objects

We have already discussed in previous tutorial, a class is a virtual entity and can be seen as a blueprint of an object. The class came into existence when it instantiated. Let's understand it by an example.
Suppose a class is a prototype of a building. A building contains all the details about the floor, rooms, doors, windows, etc. we can make as many buildings as we want, based on these details. Hence, the building can be seen as a class, and we can create as many objects of this class.
On the other hand, the object is the instance of a class. The process of creating an object can be called instantiation.

In this section of the tutorial, we will discuss creating classes and objects in Python. We will also discuss how a class attribute is accessed by using the object.

Creating classes in Python

In Python, a class can be created by using the keyword class, followed by the class name. The syntax to create a class is given below.

Syntax

```
class ClassName:
    #statement_suite
```

In Python, we must notice that each class is associated with a documentation string which can be accessed by using **<class-name>.__doc__.** A class contains a statement suite including fields, constructor, function, etc. definition.

Consider the following example to create a class **Employee** which contains two fields as Employee id, and name.

The class also contains a function **display()**, which is used to display the information of the **Employee.**

Example

```
class Employee:
        id = 10
    name = "Devansh"
    def display (self):
        print(self.id,self.name)
```

Here, the **self** is used as a reference variable, which refers to the current class object. It is always the first argument in the function definition. However, using **self** is optional in the function call.

The self-parameter

The self-parameter refers to the current instance of the class and accesses the class variables. We can use anything instead of self, but it must be the first parameter of any function which belongs to the class.

Creating an instance of the class

A class needs to be instantiated if we want to use the class attributes in another class or method. A class can be instantiated by calling the class using the class name.

The syntax to create the instance of the class is given below.

```
<object-name> = <class-name>(<arguments>)
```

The following example creates the instance of the class Employee defined in the above example.

Example

```
class Employee:
    id = 10
    name = "John"
    def display (self):
        print("ID: %d \nName: %s"%(self.id,self.name))
# Creating a emp instance of Employee class
emp = Employee()
emp.display()
```

Output:

```
ID: 10
Name: John
```

In the above code, we have created the Employee class which has two attributes named id and name and assigned value to them. We can observe we have passed the self as parameter in display function. It is used to refer to the same class attribute.
We have created a new instance object named **emp.** By using it, we can access the attributes of the class.

Delete the Object

We can delete the properties of the object or object itself by using the del keyword. Consider the following example.

Example

```
class Employee:
    id = 10
    name = "John"

    def display(self):
        print("ID: %d \nName: %s" % (self.id, self.name))
    # Creating a emp instance of Employee class
  emp = Employee()
  # Deleting the property of object
del emp.id
# Deleting the object itself
del emp
emp.display()
```

It will through the Attribute error because we have deleted the object **emp**.

Python Constructor

A constructor is a special type of method (function) which is used to initialize the instance members of the class.
In C++ or Java, the constructor has the same name as its class, but it treats constructor differently in Python. It is used to create an object.
Constructors can be of two types.

1. Parameterized Constructor
2. Non-parameterized Constructor

Constructor definition is executed when we create the object of this class. Constructors also verify that there are enough resources for the object to perform any start-up task.
Creating the constructor in python

In Python, the method the **__init__()** simulates the constructor of the class. This method is called when the class is instantiated. It accepts the **self**-keyword as a first argument which allows accessing the attributes or method of the class.
We can pass any number of arguments at the time of creating the class object, depending upon the **__init__()** definition. It is mostly used to initialize the class attributes. Every class must have a constructor, even if it simply relies on the default constructor.
Consider the following example to initialize the **Employee** class attributes.

Example

```
class Employee:
    def __init__(self, name, id):
        self.id = id
        self.name = name
      def display(self):
        print("ID: %d \nName: %s" % (self.id, self.name))
   emp1 = Employee("John", 101)
emp2 = Employee("David", 102)
  # accessing display() method to print employee 1 information
  emp1.display()
   accessing display() method to print employee 2 information
emp2.display()
```

Output:

```
ID: 101
Name: John
ID: 102
Name: David
```

Counting the number of objects of a class

The constructor is called automatically when we create the object of the class. Consider the following example.

Example

```
class Student:
    count = 0
    def __init__(self):
        Student.count = Student.count + 1
s1=Student()
s2=Student()
s3=Student()
print("The number of students:",Student.count)
```

Output:

```
The number of students: 3
```

Python Non-Parameterized Constructor

The non-parameterized constructor uses when we do not want to manipulate the value or the constructor that has only self as an argument. Consider the following example.

Example

```
class Student:
    # Constructor - non parameterized
    def __init__(self):
        print("This is non parametrized constructor")
    def show(self,name):
        print("Hello",name)
student = Student()
student.show("John")
```

Python Parameterized Constructor

The parameterized constructor has multiple parameters along with the **self**. Consider the following example.

Example

```
class Student:
    # Constructor - parameterized
    def __init__(self, name):
        print("This is parametrized constructor")
        self.name = name
    def show(self):
        print("Hello",self.name)
student = Student("John")
student.show()
```

Output:

```
This is parametrized constructor
Hello John
```

Python Default Constructor

When we do not include the constructor in the class or forget to declare it, then that becomes the default constructor. It does not perform any task but initializes the objects. Consider the following example.

Example

```
class Student:
    roll_num = 101
    name = "Joseph"
      def display(self):
        print(self.roll_num,self.name)
```

```
    st = Student()
st.display()
```

Output:

101 Joseph

More than One Constructor in Single class

Let's have a look at another scenario, what happen if we declare the two same constructors in the class.

Example

```
class Student:
    def __init__(self):
        print("The First Constructor")
    def __init__(self):
        print("The second contructor")


st = Student()
```

Output:

The Second Constructor

In the above code, the object **st** called the second constructor whereas both have the same configuration. The first method is not accessible by the **st** object. Internally, the object of the class will always call the last constructor if the class has multiple constructors.

Note: The constructor overloading is not allowed in Python.

Python built-in class functions

The built-in functions defined in the class are described in the following table.

SN	Function	Description
1	getattr(obj,name,default)	It is used to access the attribute of the object.
2	setattr(obj, name,value)	It is used to set a particular value to the specific attribute of an object.
3	delattr(obj, name)	It is used to delete a specific attribute.
4	hasattr(obj, name)	It returns true if the object contains some specific attribute.

Example

```
class Student:
 def __init__(self, name, id, age):
```

```
        self.name = name
         self.id = id
         self.age = age

         # creates the object of the class Student
    s = Student("John", 101, 22)
      # prints the attribute name of the object s
    print(getattr(s, 'name'))
      # reset the value of attribute age to 23
    setattr(s, "age", 23)
      # prints the modified value of age
    print(getattr(s, 'age'))
      # prints true if the student contains the attribute with name id
      print(hasattr(s, 'id'))
    # deletes the attribute age
    delattr(s, 'age')
      # this will give an error since the attribute age has been deleted

    print(s.age)
```

Output:

```
John
23
True
AttributeError: 'Student' object has no attribute 'age'
```

Built-in class attributes

Along with the other attributes, a Python class also contains some built-in class attributes which provide information about the class.

The built-in class attributes are given in the below table.

SN	Attribute	Description
1	__dict__	It provides the dictionary containing the information about the class namespace.
2	__doc__	It contains a string which has the class documentation
3	__name__	It is used to access the class name.
4	__module__	It is used to access the module in which, this class is defined.

5	__bases__	It contains a tuple including all base classes.

Example

```
class Student:
    def __init__(self,name,id,age):
        self.name = name;
        self.id = id;
        self.age = age
    def display_details(self):
        print("Name:%s, ID:%d, age:%d"%(self.name,self.id))
s = Student("John",101,22)
print(s.__doc__)
print(s.__dict__)
print(s.__module__)
```

Output:

```
None
{'name': 'John', 'id': 101, 'age': 22}
__main__
```

Abstraction in Python

Abstraction is used to hide the internal functionality of the function from the users. The users only interact with the basic implementation of the function, but inner working is hidden. User is familiar with that "**what function does**" but they don't know "**how it does.**"
In simple words, we all use the smartphone and very much familiar with its functions such as camera, voice-recorder, call-dialing, etc., but we don't know how these operations are happening in the background. Let's take another example - When we use the TV remote to increase the volume. We don't know how pressing a key increases the volume of the TV. We only know to press the "+" button to increase the volume.
That is exactly the abstraction that works in the object-oriented concept.

Why Abstraction is Important?

In Python, an abstraction is used to hide the irrelevant data/class in order to reduce the complexity. It also enhances the application efficiency. Next, we will learn how we can achieve abstraction using the Python program.

Abstraction classes in Python

In Python, abstraction can be achieved by using abstract classes and interfaces.
A class that consists of one or more abstract method is called the abstract class. Abstract methods do not contain their implementation. Abstract class can be inherited by the subclass and abstract method gets its definition in the subclass. Abstraction classes are meant to be the blueprint of the other class.
An abstract class can be useful when we are designing large functions. An abstract class is also

helpful to provide the standard interface for different implementations of components. Python provides the **abc** module to use the abstraction in the Python program. Let's see the following syntax.

Syntax

```
from abc import ABC
class ClassName(ABC):
```

We import the ABC class from the **abc** module.

Abstract Base Classes

An abstract base class is the common application program of the interface for a set of subclasses. It can be used by the third-party, which will provide the implementations such as with plugins. It is also beneficial when we work with the large code-base hard to remember all the classes.

Working of the Abstract Classes

Unlike the other high-level language, Python doesn't provide the abstract class itself. We need to import the abc module, which provides the base for defining Abstract Base classes (ABC). The ABC works by decorating methods of the base class as abstract. It registers concrete classes as the implementation of the abstract base. We use the ***@abstractmethod*** decorator to define an abstract method or if we don't provide the definition to the method, it automatically becomes the abstract method. Let's understand the following example.

Example -

```
# Python program demonstrate
# abstract base class work
from abc import ABC, abstractmethod
class Car(ABC):
    def mileage(self):
        pass

class Tesla(Car):
    def mileage(self):
        print("The mileage is 30kmph")
class Suzuki(Car):
    def mileage(self):
        print("The mileage is 25kmph ")
class Duster(Car):
     def mileage(self):
          print("The mileage is 24kmph ")

class Renault(Car):
    def mileage(self):
            print("The mileage is 27kmph ")

# Driver code
t= Tesla ()
t.mileage()
  r = Renault()
r.mileage()
  s = Suzuki()
s.mileage()
d = Duster()
```

```
d.mileage()
```

Output:

```
The mileage is 30kmph
The mileage is 27kmph
The mileage is 25kmph
The mileage is 24kmph
```

Explanation -

In the above code, we have imported the **abc module** to create the abstract base class. We created the Car class that inherited the ABC class and defined an abstract method named mileage(). We have then inherited the base class from the three different subclasses and implemented the abstract method differently. We created the objects to call the abstract method.

Let's understand another example.

Let's understand another example.

Example -

```
# Python program to define
# abstract class

from abc import ABC
  class Polygon(ABC):
     # abstract method
   def sides(self):
      pass
  class Triangle(Polygon):
   def sides(self):
      print("Triangle has 3 sides")
  class Pentagon(Polygon):

     def sides(self):
      print("Pentagon has 5 sides")

class Hexagon(Polygon):
     def sides(self):
      print("Hexagon has 6 sides")

class square(Polygon):
     def sides(self):
      print("I have 4 sides")

# Driver code
t = Triangle()
t.sides()

s = square()
s.sides()

p = Pentagon()
p.sides()
```

```
k = Hexagon()
K.sides()
```

Output:

```
Triangle has 3 sides
Square has 4 sides
Pentagon has 5 sides
Hexagon has 6 sides
```

Explanation -
In the above code, we have defined the abstract base class named Polygon and we also defined the abstract method. This base class inherited by the various subclasses. We implemented the abstract method in each subclass. We created the object of the subclasses and invoke the **sides()** method. The hidden implementations for the **sides()** method inside the each subclass comes into play. The abstract method **sides()** method, defined in the abstract class, is never invoked.

Points to Remember

Below are the points which we should remember about the abstract base class in Python.

- An Abstract class can contain the both method normal and abstract method.
- An Abstract cannot be instantiated; we cannot create objects for the abstract class.

Abstraction is essential to hide the core functionality from the users. We have covered the all the basic concepts of Abstraction in Python.

Encapsulation

- When classes are defined, programmers can specify that certain methods or state variables remain hidden inside the class.
- These variables and methods are accessible from within the class, but not accessible outside it.
- The combination of collecting all the attributes of an object into a single class definition, combined with the ability to hide some definitions and type information within the class, is known as encapsulation.
- Using OOP in Python, we can restrict access to methods and variables. This prevents data from direct modification which is called encapsulation.
- This can prevent the data from being modified by accident and is known as encapsulation
- In Python, we denote private attributes using underscore as the prefix i.e single _ or double __.

Polymorphism in Python

In this tutorial, we will learn about polymorphism, different types of polymorphism, and how we can implement them in Python with the help of examples.

What is Polymorphism?

The literal meaning of polymorphism is the condition of occurrence in different forms.

Polymorphism is a very important concept in programming. It refers to the use of a single type entity (method, operator or object) to represent different types in different scenarios.

Let's take an example:

Example 1: Polymorphism in addition operator

We know that the + operator is used extensively in Python programs. But, it does not have a single usage.

For integer data types, + operator is used to perform arithmetic addition operation.

```
num1 = 1
num2 = 2
print(num1+num2)
```

Hence, the above program outputs 3.

Similarly, for string data types, + operator is used to perform concatenation.

```
str1 = "Python"
str2 = "Programming"
print(str1+" "+str2)
```

As a result, the above program outputs Python Programming.

Here, we can see that a single operator + has been used to carry out different operations for distinct data types. This is one of the most simple occurrences of polymorphism in Python.

Function Polymorphism in Python

There are some functions in Python which are compatible to run with multiple data types.

One such function is the len() function. It can run with many data types in Python. Let's look at some example use cases of the function.

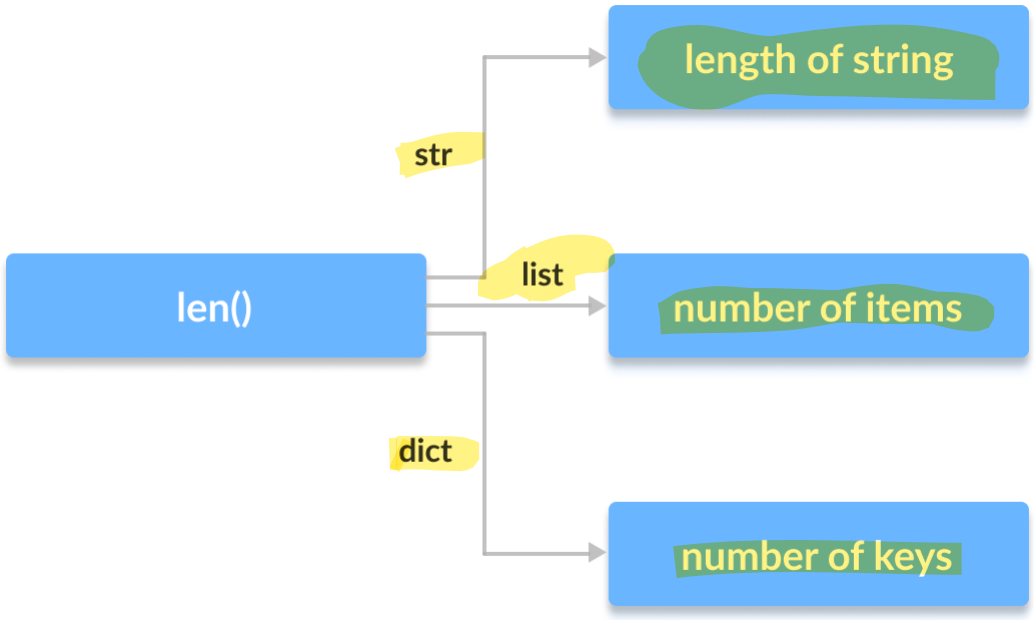
Example 2: Polymorphic len() function

```
print(len("Programiz"))
print(len(["Python", "Java", "C"]))
print(len({"Name": "John", "Address": "Nepal"}))
```

Output

```
9
3
2
```

Here, we can see that many data types such as string, list, tuple, set, and dictionary can work with the len() function. However, we can see that it returns specific information about specific data types.



Polymorphism in len() function in Python

Class Polymorphism in Python

Polymorphism is a very important concept in Object-Oriented Programming.

To learn more about OOP in Python, visit: Python Object-Oriented Programming

We can use the concept of polymorphism while creating class methods as Python allows different classes to have methods with the same name.

We can then later generalize calling these methods by disregarding the object we are working with.

Let's look at an example:

Example 3: Polymorphism in Class Methods

```
class Cat:
    def __init__(self, name, age):
        self.name = name
```

```
        self.age = age

    def info(self):
        print(f"I am a cat. My name is {self.name}. I am {self.age} years old.")

    def make_sound(self):
        print("Meow")


class Dog:
    def __init__(self, name, age):
        self.name = name
        self.age = age

    def info(self):
        print(f"I am a dog. My name is {self.name}. I am {self.age} years old.")

    def make_sound(self):
        print("Bark")


cat1 = Cat("Kitty", 2.5)
dog1 = Dog("Fluffy", 4)

for animal in (cat1, dog1):
    animal.make_sound()
    animal.info()
    animal.make_sound()
```

Output

```
Meow
I am a cat. My name is Kitty. I am 2.5 years old.
Meow
Bark
I am a dog. My name is Fluffy. I am 4 years old.
Bark
```

Here, we have created two classes Cat and Dog. They share a similar structure and have the same method names info() and make_sound().

However, notice that we have not created a common superclass or linked the classes together in any way. Even then, we can pack these two different objects into a tuple and iterate through it using a common animal variable. It is possible due to polymorphism.

OOPS Exercises:

1: Create a Vehicle class with max_speed and mileage instance attributes

Solution:

```
class Vehicle:
    def __init__(self, max_speed, mileage):
        self.max_speed = max_speed
        self.mileage = mileage

modelX = Vehicle(240, 18)
print(modelX.max_speed, modelX.mileage)
```

2: Create a Vehicle class without any variables and methods

```
class Vehicle:
    pass
```

Create a child class Bus that will inherit all of the variables and methods of the Vehicle class

```
class Vehicle:

    def __init__(self, name, max_speed, mileage):
        self.name = name
        self.max_speed = max_speed
        self.mileage = mileage
```

Create a Bus object that will inherit all of the variables and methods of the Vehicle class and display it.

```
class Vehicle:
```

Solution:

```
    def __init__(self, name, max_speed, mileage):
        self.name = name
        self.max_speed = max_speed
        self.mileage = mileage

class Bus(Vehicle):
    pass

School_bus = Bus("School Volvo", 180, 12)
print("Vehicle Name:", School_bus.name, "Speed:", School_bus.max_speed, "Mileage:",
School_bus.mileage)
```

1. """Question:

Define a class, which have a class parameter and have a same instance parameter.

Hints:
Define a instance parameter, need add it in __init__ method
You can init a object with construct parameter or set the value later

Solution:

```
"""
class Person:
    # Define the class parameter "name"
    name = "Person"

    def __init__(self, name = None):
        # self.name is the instance parameter
        self.name = name

jeffrey = Person("Jeffrey")
print "%s name is %s" % (Person.name, jeffrey.name)

nico = Person()
nico.name = "Nico"
print "%s name is %s" % (Person.name, nico.name)
```

7 """Define a class named Rectangle which can be constructed by a length and width. The Rectangle class has a method which can compute the area.

Hints:

Use def methodName(self) to define a method.

Solution:

```
"""
class Rectangle(object):
    def __init__(self, l, w):
        self.length = l
        self.width  = w

    def area(self):
        return self.length*self.width

aRectangle = Rectangle(2,10)
print aRectangle.area()
```

- Write a Python class named Student with two attributes student_id, student_name. Add a new attribute student_class. Create a function to display the entire attribute and their values in Student class.

```
class Student:
    student_id = 'V10'
    student_name = 'Jacqueline Barnett'
```

```
    def display():
        print(f'Student id: {Student.student_id}\nStudent Name: {Student.student_name}')
print("Original attributes and their values of the Student class:")
Student.display()
```

```
Original attributes and their values of the Student class:
Student id: V10
Student Name: Jacqueline Barnett
```

Import built-in array module and display the namespace of the said module

```
import array
for name in array.__dict__:
    print(name)
```

Sample Output:

```
__name__
__doc__
__package__
__loader__
__spec__
_array_reconstructor
ArrayType
array
typecodes
```

- Write a Python program to create a class and display the namespace of the said class.

Sample Solution:

Python Code:

```
class py_solution:
    def sub_sets(self, sset):
        return self.subsetsRecur([], sorted(sset))
    def subsetsRecur(self, current, sset):
        if sset:
            return self.subsetsRecur(current, sset[1:]) + self.subsetsRecur(current + [sset[0]], sset[1:])
        return [current]
for name in py_solution.__dict__:
    print(name)
```

Copy

Sample Output:

```
__module__
sub_sets
subsetsRecur
__dict__
__weakref__
```

```
__doc__
```

- Write a Python program to create an instance of a specified class and display the namespace of the said instance.

Sample Solution:

Python Code:

```
class Student:
    def __init__(self, student_id, student_name, class_name):
        self.student_id = student_id
        self.student_name = student_name
        self.class_name = class_name
student = Student('V12', 'Frank Gibson', 'V')
print(student.__dict__)
```

Copy

Sample Output:

```
{'student_id': 'V12', 'student_name': 'Frank Gibson', 'class_name': 'V'}
```

- Write a Python class named Student with two attributes student_name, marks. Modify the attribute values of the said class and print the original and modified values of the said attributes.

Solution:

Python Code:

```
class Student:
    student_name = 'Terrance Morales'
    marks = 93
print(f"Student Name: {getattr(Student, 'student_name')}")
print(f"Marks: {getattr(Student, 'marks')}")
setattr(Student, 'student_name', 'Angel Brooks')
setattr(Student, 'marks', 95)
print(f"Student Name: {getattr(Student, 'student_name')}")
print(f"Marks: {getattr(Student, 'marks')}")
```

Copy

Sample Output:

```
Student Name: Terrance Morales
Marks: 93
Student Name: Angel Brooks
Marks: 95
```

- Write a Python class named Student with two instances student1, student2 and assign given values to the said instances attributes. Print all the attributes of student1, student2 instances with their values in the given format.

```
class Student:
    school = 'Forward Thinking'
    address = '2626/Z Overlook Drive, COLUMBUS'
student1 = Student()
student2 = Student()
student1.student_id = "V12"
student1.student_name = "Ernesto Mendez"
student2.student_id = "V12"
student2.marks_language = 85
student2.marks_science = 93
student2.marks_math = 95
students = [student1, student2]
for student in students:
    print('\n')
    for attr in student.__dict__:
        print(f'{attr} -> {getattr(student, attr)}')
```

```
student_id -> V12
student_name -> Ernesto Mendez


student_id -> V12
marks_language -> 85
marks_science -> 93
marks_math -> 95
```

- Write a Python program to convert an integer to a roman numeral.

Solution:

Python Code:

```
class py_solution:
    def int_to_Roman(self, num):
        val = [
            1000, 900, 500, 400,
            100, 90, 50, 40,
            10, 9, 5, 4,
            1
            ]
        syb = [
            "M", "CM", "D", "CD",
            "C", "XC", "L", "XL",
            "X", "IX", "V", "IV",
            "I"
            ]
        roman_num = ''
        i = 0
        while  num > 0:
```

```
            for _ in range(num // val[i]):
                roman_num += syb[i]
                num -= val[i]
            i += 1
        return roman_num
print(py_solution().int_to_Roman(1))
print(py_solution().int_to_Roman(4000))
```

Copy
Sample Output:

```
I
MMMM
```

Write a Python program to convert an integer to a roman numeral.

Solution:

Python Code:

```
class py_solution:
    def int_to_Roman(self, num):
        val = [
            1000, 900, 500, 400,
            100, 90, 50, 40,
            10, 9, 5, 4,
            1
            ]
        syb = [
            "M", "CM", "D", "CD",
            "C", "XC", "L", "XL",
            "X", "IX", "V", "IV",
            "I"
            ]
        roman_num = ''
        i = 0
        while  num > 0:
            for _ in range(num // val[i]):
                roman_num += syb[i]
                num -= val[i]
            i += 1
        return roman_num


print(py_solution().int_to_Roman(1))
print(py_solution().int_to_Roman(4000))
```

Output:

```
I
MMMM
```

Write a Python program to find the three elements that sum to zero from a set (array) of n real numbers.

Solution:

Python Code:

```
class py_solution:
 def threeSum(self, nums):
       nums, result, i = sorted(nums), [], 0
       while i < len(nums) - 2:
          j, k = i + 1, len(nums) - 1
          while j < k:
             if nums[i] + nums[j] + nums[k] < 0:
                j += 1
             elif nums[i] + nums[j] + nums[k] > 0:
                k -= 1
             else:
                result.append([nums[i], nums[j], nums[k]])
                j, k = j + 1, k - 1
                while j < k and nums[j] == nums[j - 1]:
                   j += 1
                while j < k and nums[k] == nums[k + 1]:
                   k -= 1
          i += 1
          while i < len(nums) - 2 and nums[i] == nums[i - 1]:
             i += 1
       return result

print(py_solution().threeSum([-25, -10, -7, -3, 2, 4, 8, 10]))
```

Output:

```
[[-10, 2, 8], [-7, -3, 10]]
```

Write a Python program to implement pow(x, n).

Solution:

Python Code:

```
class py_solution:
  def pow(self, x, n):
      if x==0 or x==1 or n==1:
         return x
      if x==-1:
         if n%2 ==0:
            return 1
         else:
            return -1
      if n==0:
         return 1
      if n<0:
```

```
            return 1/self.pow(x,-n)
        val = self.pow(x,n//2)
        if n%2 ==0:
            return val*val
        return val*val*x

print(py_solution().pow(2, -3));
print(py_solution().pow(3, 5));
print(py_solution().pow(100, 0));
```

Output:

```
0.125
243
1
```

Write a Python program to reverse a string word by word.

Solution:

Python Code:

```
class py_solution:
    def reverse_words(self, s):
        return ' '.join(reversed(s.split()))


print(py_solution().reverse_words('hello .py'))
```

Output:

```
.py hello
```

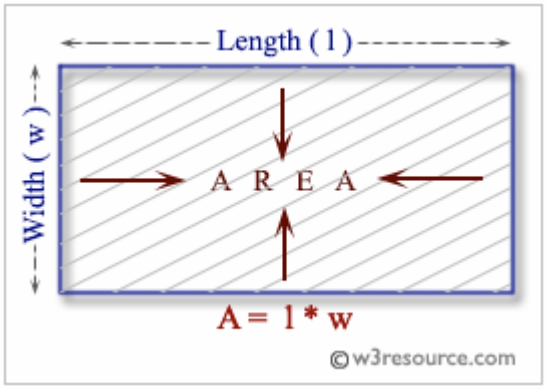
- Write a Python class named Rectangle constructed by a length and width and a method which will compute the area of a rectangle.

Area of a rectangle

In Euclidean plane geometry, a rectangle is a quadrilateral with four right angles. To find the area of a rectangle, multiply the length by the width.

A rectangle with four sides of equal length is a square.

Following image represents the area of a rectangle.



Solution:

Python Code:

```
class Rectangle():
    def __init__(self, l, w):
        self.length = l
        self.width  = w

    def rectangle_area(self):
        return self.length*self.width

newRectangle = Rectangle(12, 10)
print(newRectangle.rectangle_area())
```

Output:

```
120
```

Write a Python class named Circle constructed by a radius and two methods which will compute the area and the perimeter of a circle.

Sample Solution:

Python Code:

```
class Circle():
    def __init__(self, r):
        self.radius = r

    def area(self):
        return self.radius**2*3.14

    def perimeter(self):
        return 2*self.radius*3.14

NewCircle = Circle(8)
print(NewCircle.area())
print(NewCircle.perimeter())
```

Output:

```
200.96
50.24
```

Inheritance:

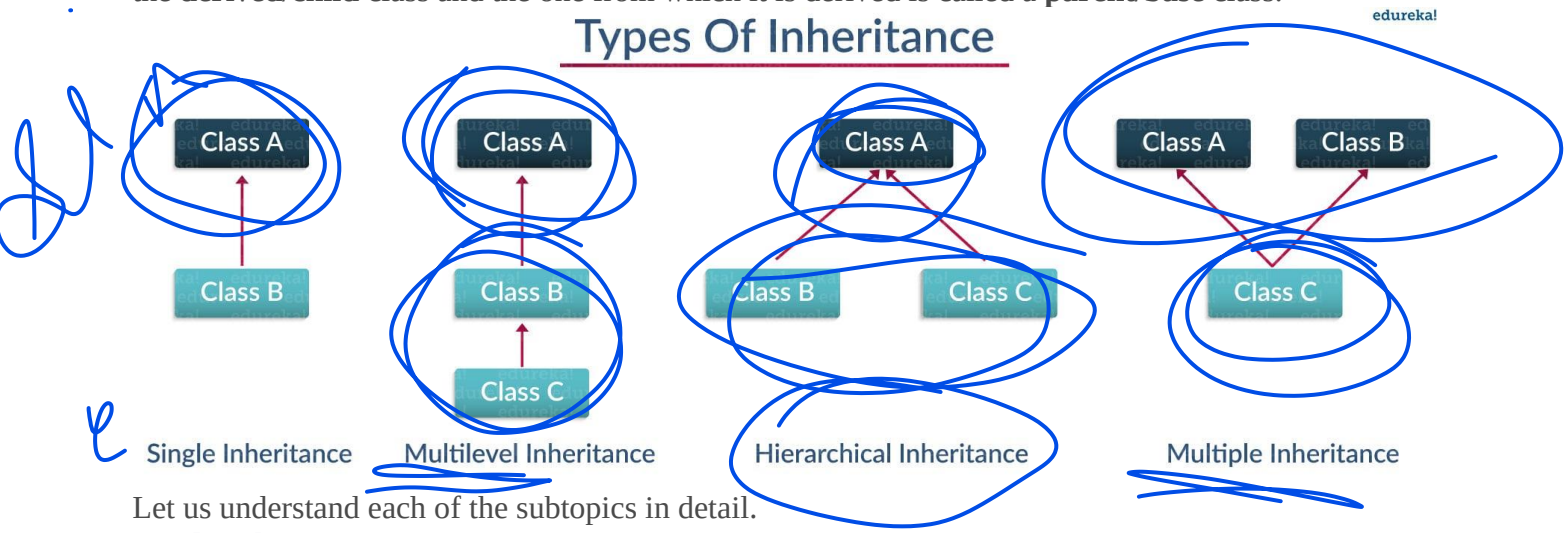
Inheritance enables us to define a class that takes all the functionality from a parent class and allows us to add more. It refers to defining a new class with little or no modification to an existing class. The new class is called derived (or child) class and the one from which it inherits is called the base (or parent) class.

Python Inheritance Syntax

```
class BaseClass:
  Body of base class
class DerivedClass(BaseClass):
  Body of derived class
```

Derived class inherits features from the base class where new features can be added to it. This results in re-usability of code.

Ever heard of this dialogue from relatives "you look exactly like your father/mother" the reason behind this is called 'inheritance'. From the Programming aspect, It generally means "inheriting or transfer of characteristics from parent to child class without any modification". The new class is called the **derived/child** class and the one from which it is derived is called a **parent/base** class.



Let us understand each of the subtopics in detail.

Single Inheritance:

Single level inheritance enables a derived class to inherit characteristics from a single parent class.

Example:

```
class employee1()://This is a parent class
```

```
        def __init__(self, name, age, salary):
        self.name = name
        self.age = age
        self.salary = salary
        class childemployee(employee1)://This is a child class
        def __init__(self, name, age, salary,id):
        self.name = name
        self.age = age
        self.salary = salary
        self.id = id
        emp1 = employee1('harshit',22,1000)
        print(emp1.age)
```

Output: 22

Explanation:

- I am taking the parent class and created a constructor (__init__), class itself is initializing the attributes with parameters('name', 'age' and 'salary').
- Created a child class 'childemployee' which is inheriting the properties from a parent class and finally instantiated objects 'emp1' and 'emp2' against the parameters.
- Finally, I have printed the age of emp1. Well, you can do a hell lot of things like print the whole dictionary or name or salary.

Example:

A polygon is a closed figure with 3 or more sides. Say, we have a class called Polygon defined as follows

```
class Polygon:
    def __init__(self, no_of_sides):
        self.n = no_of_sides
        self.sides = [0 for i in range(no_of_sides)]

    def inputSides(self):
        self.sides = [float(input("Enter side "+str(i+1)+" : ")) for i in range(self.n)]

    def dispSides(self):
        for i in range(self.n):
            print("Side",i+1,"is",self.sides[i])
```

A triangle is a polygon with 3 sides. So, we can create a class called Triangle which inherits from Polygon. This makes all the attributes of Polygon class available to the Triangle class. We don't need to define them again (code reusability). Triangle can be defined as follows.

```
class Triangle(Polygon):
    def __init__(self):
        Polygon.__init__(self,3)

    def findArea(self):
        a, b, c = self.sides
        # calculate the semi-perimeter
        s = (a + b + c) / 2
        area = (s*(s-a)*(s-b)*(s-c)) ** 0.5
        print('The area of the triangle is %0.2f' %area)
```

However, class Triangle has a new method findArea() to find and print the area of the triangle.

Here is a sample run.

```
>>> t = Triangle()

>>> t.inputSides()
Enter side 1 : 3
Enter side 2 : 5
Enter side 3 : 4

>>> t.dispSides()
Side 1 is 3.0
Side 2 is 5.0
Side 3 is 4.0

>>> t.findArea()
The area of the triangle is 6.00
```

Benefits of inheritance

1. It represents real-world relationships well.
2. It provides reusability of a code. We don't have to write the same code again and again. Also, it allows us to add more features to a class without modifying it.
3. It is transitive in nature, which means that if class B inherits from another class A, then all the subclasses of B would automatically inherit from class A.

Files

File is a named location on disk to store related information. It is used to permanently store data in a non-volatile memory (E.g. Hard disk) . Python too supports file handling and allows users to handle files i.e., to read and write files, along with many other file handling options, to operate on files. Python treats file differently as text or binary. Each line of code includes a sequence of characters and they form text file. Each line of a file is terminated with a special character, called the EOL or End of Line characters like comma {,} or newline character.

[What is a text file? Give an example for a text file.]

Python provides inbuilt functions for creating, writing and reading files. There are two types of files that can be handled in python, normal text files and binary files (written in binary language,0s and 1s).

- Text files: Text files are structured as a sequence of lines, where each line includes a sequence of characters. This is what We know as code or syntax. In this type of file, Each line of text is terminated with a special character called EOL (End of Line), which is the new line character (‘\n’) in python by default.

- Binary files: In this type of file, there is no terminator for a line and the data is stored after converting it into machine understandable binary language.

Python has several functions for creating, reading, updating, and deleting files. File operation takes place in the following order.

- Open a file
- Read or write (perform operation)
- Close the file

The open Function

[How to open a new file in Python?]

Python has a built-in open() function to open a file. This function returns a file object, also called a handle and it is used to read or modify the file accordingly.

```
file object = open(<file-name>, <access-mode>, <buffering>)
```

- file_name − The filename argument is a string value that contains the name of the file that We want to access.

- access_mode- The access mode determines the mode in which the file has to be opened, i.e., read, write, append, etc. This is optional parameter, and the default file access mode is read (r).
- Buffering- If the buffering value is set to 0, no buffering takes place. If the buffering value is 1, line buffering is performed while accessing a file. If We specify the buffering value as an integer greater than 1, then buffering action is performed with the indicated buffer size. If negative, the buffer size is the system default(default behavior).

There are four different methods (modes) for opening a file:

"r" - Read - Default value. Opens a file for reading, error if the file does not exist.

"a" - Append - Opens a file for appending, creates the file if it does not exist.

"w" - Write - Opens a file for writing, creates the file if it does not exist.

"x" - Create - Creates the specified file, returns an error if the file exists.

"+" - Open a file for updating (reading and Writing)

"r+" - for both reading and writing

In addition, We can specify if the file should be handled as binary or text mode.

"t" - Text - Default value. Text mode

"b" - Binary - Binary mode (e.g. images)

In Detail all access modes are given in below table

r	It opens the file to read-only mode. The file pointer exists at the beginning. The file is by default open in this mode if no access mode is passed.
rb	It opens the file to read-only in binary format. The file pointer exists at the beginning of the file.
r+	It opens the file to read and write both. The file pointer exists at the beginning of the file.
rb+	It opens the file to read and write both in binary format. The file pointer exists at the beginning of the file.

w	It opens the file to write only. It overwrites the file if previously exists or creates a new one if no file exists with the same name. The file pointer exists at the beginning of the file.
wb	It opens the file to write only in binary format. It overwrites the file if it exists previously or creates a new one if no file exists. The file pointer exists at the beginning of the file.
w+	It opens the file to write and read both. It is different from r+ in the sense that it overwrites the previous file if one exists whereas r+ doesn't overwrite the previously written file. It creates a new file if no file exists. The file pointer exists at the beginning of the file.
wb+	It opens the file to write and read both in binary format. The file pointer exists at the beginning of the file.
a	It opens the file in the append mode. The file pointer exists at the end of the previously written file if exists any. It creates a new file if no file exists with the same name.
ab	It opens the file in the append mode in binary format. The pointer exists at the end of the previously written file. It creates a new file in binary format if no file exists with the same name.
a+	It opens a file to append and read both. The file pointer remains at the end of the file if a file exists. It creates a new file if no file exists with the same name.
ab+	It opens a file to append and read both in binary format. The file pointer remains at the end of the file.

We can specify the mode while opening a file. In mode, we specify whether we want to read r, write w or append a to the file. We can also specify if we want to open the file in text mode or binary mode.

Examples:

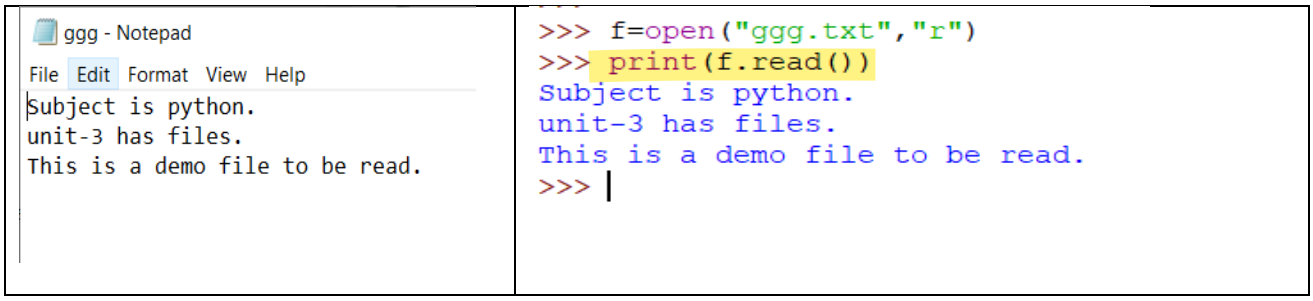
Commands	Meaning
f = open("test.txt")	#open file in current directory # equivalent to 'r' or 'rt'
f = open("C:/Python38/README.txt")	#specifying full path
f = open("test.txt",'w')	# write in text mode
f = open("img.bmp",'r+b')	#read and write in binary mode

f = open("demofile.txt", "rt")	#Because "r" for read, and "t" for text are the default values, We do not need to specify them.
file = open('geek.txt', 'r')	#file named "geek", will be opened with the reading mode.

The read Function

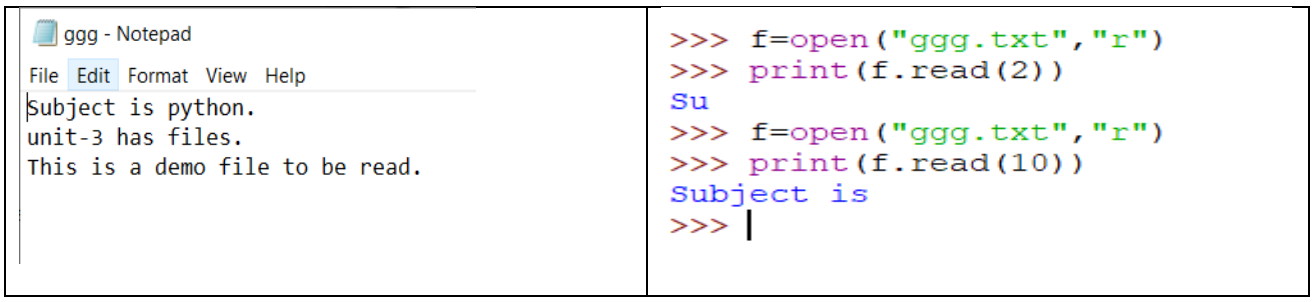
To read a file in Python, we must open the file in reading **r** mode. There are various methods available for this purpose. We can use the **read(size)** method to read in the size number of data. If the size parameter is not specified, it reads and returns up to the end of the file.

[Which method is used to read the contents of a file which is already created?]



Read Only Parts of the File:-

By default the read() method returns the whole text, but We can also specify how many characters We want to return



Read Lines

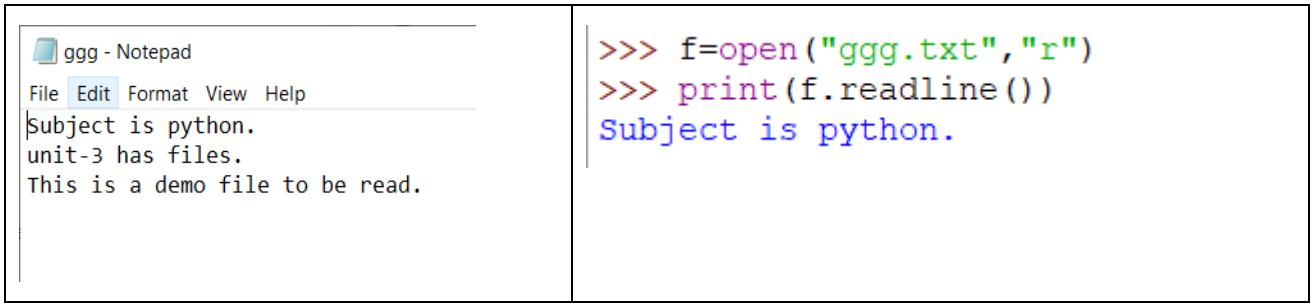
The readline() method returns one line from the file. We can also specified how many bytes from the line to return, by using the size parameter.

file.readline(*size*)

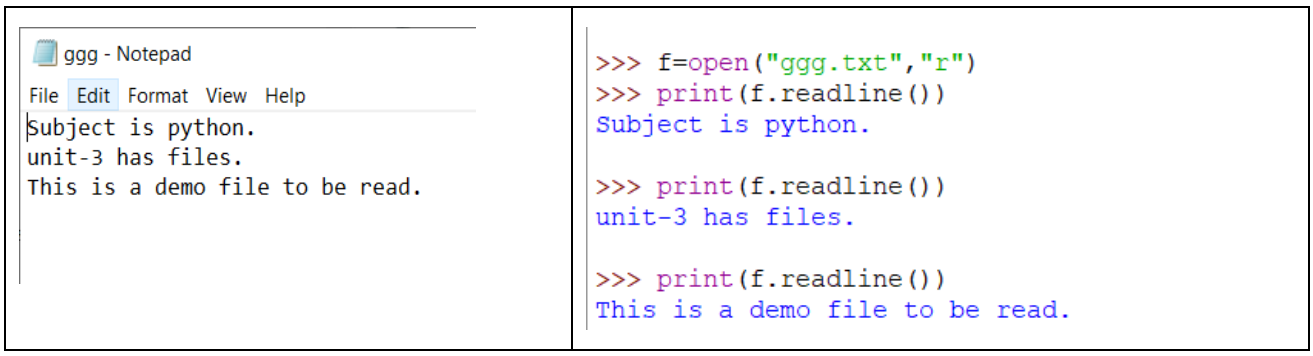
Parameter Values

Parameter	Description
size	Optional. The number of bytes from the line to return. Default -1, which means the whole line.

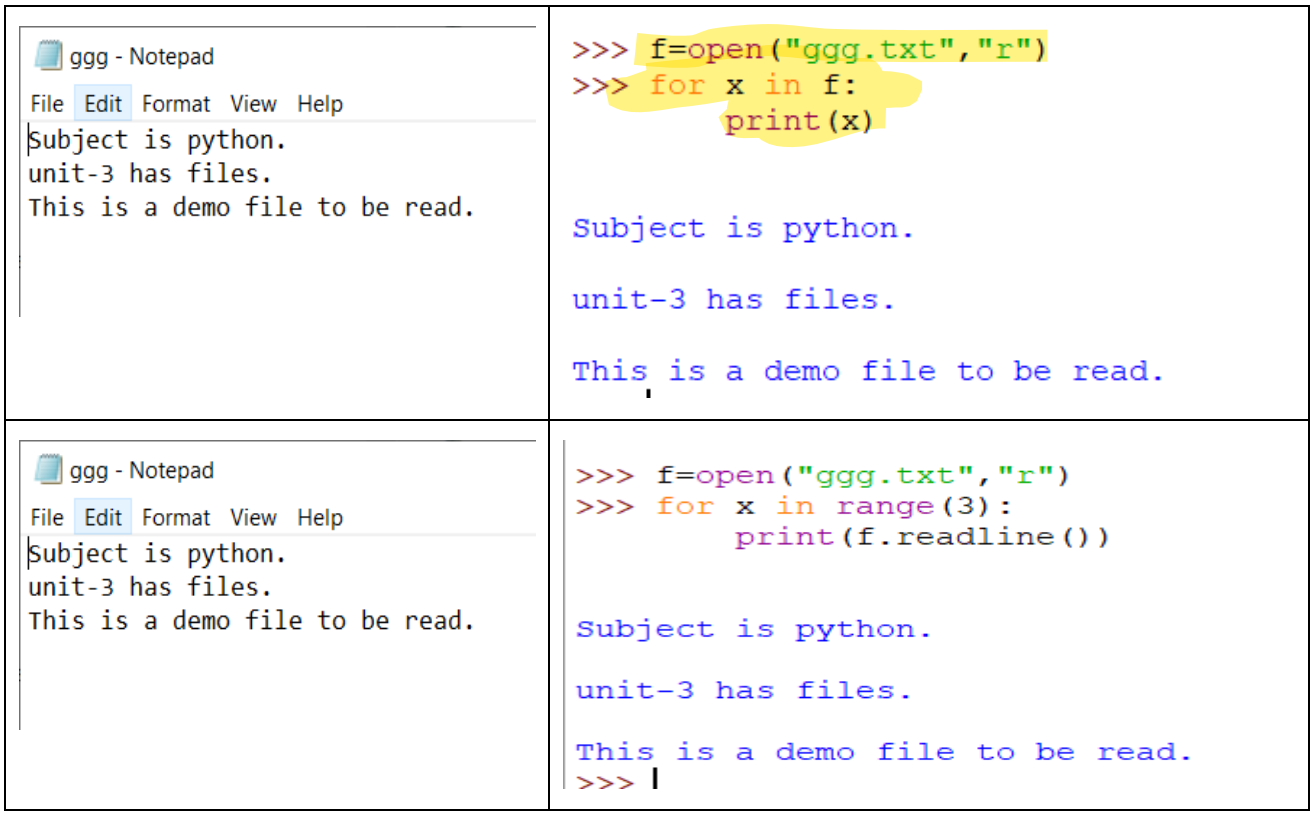
To return one line by using the readline() method:



By calling readline() two times, We can read the two first lines:



By looping through the lines of the file, we can read the whole file, line by line: This is both efficient and fast.



The readlines() method returns a list of remaining lines of the entire file. All these reading methods return empty values when the end of file (EOF) is reached.

```
f.readlines()
```

```
>>> f.seek(50)
50
>>> f.readline()
'o file to be read\n'
>>> f.readline()
'Subject is python\n'
>>> f.readlines()
['unit-3 has files\n', 'This is a demo file to be read\n']
>>>
```

ggg - Notepad

File Edit Format View Help

```
Subject is python
unit-3 has files
This is a demo file to be read
Subject is python
unit-3 has files
This is a demo file to be read
```

We can see that the read() method returns a newline as '\n'. Once the end of the file is reached, we get an empty string on further reading.

```
>>> f=open("ggg.txt","r")
>>> f.read(15)
'Subject is pyth'
>>> f.read(15)
'on\nunit-3 has f'
>>> f.read(15)
'iles\nThis is a '
>>> f.read(15)
'demo file to be'
>>> f.read(15)
' read\nSubject i'
>>> f.read(15)
's python\nunit-3'
>>> f.read(15)
' has files\nThis'
>>> f.read(15)
' is a demo file'
>>> f.read(15)
' to be read\n'
>>> f.read(15)
''
>>> f.read(15)
''
```

File Positions (Change current file cursor (position)

The tell() method tells We the current position within the file; in other words, the next read or write will occur at that many bytes from the beginning of the file.

The seek(offset[, from]) method changes the current file position. The offset argument indicates the number of bytes to be moved. The from argument specifies the reference position from where the bytes are to be moved.

If from is set to 0, it means use the beginning of the file as the reference position and 1 means use the current position as the reference position and if it is set to 2 then the end of the file would be taken as the reference position.

>>> f.tell() # get the current file position

>>> f.seek(0) # bring file cursor to initial position

>>> print(f.read()) # read the entire file

We can change our current file cursor (position) using the seek() method. Similarly, the tell() method returns our current position (in number of bytes).

```
>>> f.tell()
138
>>> f.seek(0)
0
>>> print(f.read())
Subject is python
unit-3 has files
This is a demo file to be read
Subject is python
unit-3 has files
This is a demo file to be read

>>> f.seek(50)
50
>>> print(f.read())
o file to be read
Subject is python
unit-3 has files
This is a demo file to be read
```

The close() Method:

- When we are done with performing operations on the file, we need to properly close the file.
- Closing a file will free up the resources that were tied with the file
- The close() method of a file object flushes any unwritten information and closes the file object, after which no more writing can be done.
- Python automatically closes a file when the reference object of a file is reassigned to another file.
- It is a good practice to use the close() method to close a file.
- Close the file when We are finish with it:

fileObject.close() or f.close()

```
#!/usr/bin/python

# Open a file
fo = open("foo.txt", "wb")
print "Name of the file: ", fo.name

# Close opend file
fo.close()
```

This produces the following result −

Name of the file: foo.txt

Python Write Files

[Explain how the write method works on a file]

In order to write into a file in Python, we need to open it in write w, append a or exclusive creation X mode.

Writing a string or sequence o is done using the write() method. This method returns the **number of characters** written to the file.

Write to an Existing File

To write to an existing file, must add a parameter to the open() function:

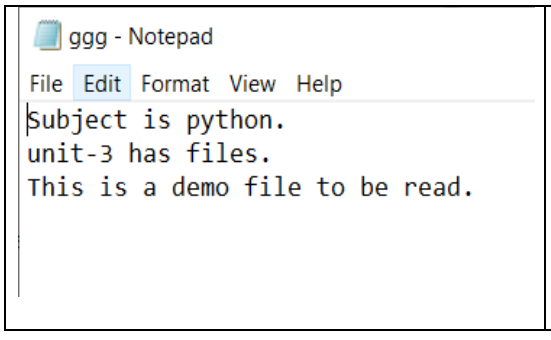
"a" - Append - will append to the end of the file

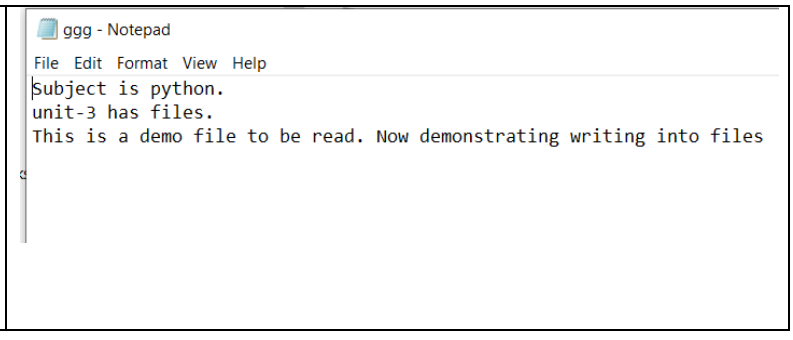
"w" - Write - will overwrite any existing content [Careful-Previous data are erased]

[How to open a new file in Python?]

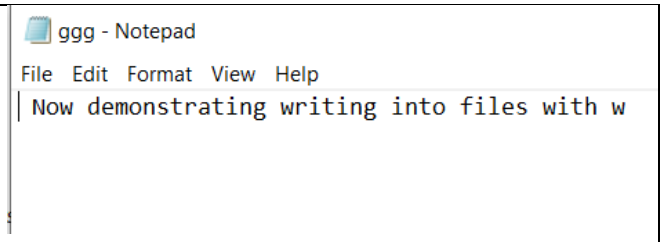
```
f = open("demofile2.txt", "a")
f.write("Now the file has more content!")
f.close()

#open and read the file after the appending:
f = open("demofile2.txt", "r")
print(f.read())
```

```
>>> f=open("ggg.txt","a")
>>> f.write(" Now demonstrating writing into files")
37
>>> f.close()
>>> f=open("ggg.txt","r")
>>> print(f.read())
Subject is python.
unit-3 has files.
This is a demo file to be read. Now demonstrating writing into files
>>>
```





```
>>> f=open("ggg.txt","w")
>>> f.write(" Now demonstrating writing into files with w")
44
>>> f.close()
>>> f=open("ggg.txt","r")
>>> print(f.read())
 Now demonstrating writing into files with w
>>>
```



This program will create a new file in the current directory if it does not exist. If it does exist, it is overwritten. We must include the newline characters ourselves to distinguish the different lines.

Write Multiple Lines

Python provides the writelines() method to save the contents of a list object in a file. Since the newline character is not automatically written to the file, it must be provided as a part of the string.

The writelines() method writes the items of a list to the file. Where the texts will be inserted depends on the file mode and stream position.

"a": The texts will be inserted at the current file stream position, default at the end of the file.

"w": The file will be emptied before the texts will be inserted at the current file stream position, default 0.

file.writelines(*list*)

Parameter	Description
list	The list of texts or byte objects that will be inserted.

```
>>> lines=["Hello world.\n", "Welcome to python class.\n"]
>>> f=open("ggg.txt", "w")
>>> f.writelines(lines)
>>> f.close()
```

Python Create Files

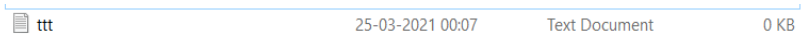
To create a new file in Python, use the open() method, with one of the following parameters:

"x" - Create - will create a file, returns an error if the file exist

"a" - Append - will create a file if the specified file does not exist

"w" - Write - will create a file if the specified file does not exist

"x" - Create - will create a file, returns an error if the file exist

Create a file called "ttt.txt": f = open("ttt.txt", "x")	
f=open("ggg.txt","x")	Traceback (most recent call last): File "<pyshell#65>", line 1, in <module> f=open("ggg.txt","x") FileExistsError: [Errno 17] File exists: 'ggg.txt' >>>

"a" - Append - will create a file if the specified file does not exist

f=open("hhh.txt","a")	>>> f=open("ttt.txt","x")
	Traceback (most recent call last):
	File "<pyshell#69>", line 1, in <module>
	f=open("ttt.txt","x")
	FileExistsError: [Errno 17] File exists: 'ttt.txt'
	>>> f=open("ttt.txt","a")
	>>> f=open("hhh.txt","a")
	>>>

"w" - Write - will create a file if the specified file does not exist

f=open("hh.txt","w")	>>> f=open("hhh.txt","w")
	>>> f=open("sss.txt","a")
	>>>

Delete a File

To delete a file, We must import the OS module, and run its os.remove() function:

import os os.remove("ggg.txt")	The file "ggg.txt"will be removed

Check if File exist:

To avoid getting an error, We might want to check if the file exists before We try to delete it:

```
import os
if os.path.exists("demofile.txt"):
  os.remove("demofile.txt")
else:
  print("The file does not exist")
```

```
>>> if os.path.exists("ggg.txt"):
        os.remove("ggg.txt")
else:
        print("The file does not exist")


The file does not exist
```

Delete Folder

To delete an entire folder, use the os.rmdir() method:

```
import os
os.rmdir("myfolder")
```

Renaming Files

The *rename()* method takes two arguments, the current filename and the new filename.

Syntax

```
os.rename(current_file_name, new_file_name)
```

Example:

```
os.rename( "test1.txt", "test2.txt" )
```

The with statement

The **with** statement was introduced in python 2.5. The with statement is useful in the case of manipulating the files. It is used in the scenario where a pair of statements is to be executed with a block of code in between.

The syntax to open a file using with the statement is given below.

```
WITH OPEN(<FILE NAME>, <ACCESS MODE>) AS <FILE-POINTER>:
    #statement suite
```

The advantage of using with statement is that it provides the guarantee to close the file regardless of how the nested block exits.

It is always suggestible to use the **with** statement in the case of files because, if the break, return, or exception occurs in the nested block of code then it automatically closes the file, we don't need to write the **close()** function. It doesn't let the file to corrupt.

Example:

```
WITH OPEN("FILE.TXT",'R') AS F:
    CONTENT = F.READ();
```

PRINT(CONTENT)

The file Object Attributes

Once a file is opened and We have one file object, We can get various information related to that file.Here is a list of all attributes related to file object –

Sr.No.	Attribute & Description
1	**file.closed** Returns true if file is closed, false otherwise.
2	**file.mode** Returns access mode with which file was opened.
3	**file.name** Returns name of the file.
4	**file.softspace** Returns false if space explicitly required with print, true otherwise.

```
#!/usr/bin/python

# Open a file

fo = open("foo.txt", "wb")

print "Name of the file: ", fo.name

print "Closed or not : ", fo.closed

print "Opening mode : ", fo.mode

print "Softspace flag : ", fo.softspace
```

This produces the following result −

Name of the file: foo.txt

Closed or not : False

Opening mode : wb

Softspace flag : 0

Python File Methods

There are various methods available with the file object. Some of them have been used in the above examples.Here is the complete list of methods in text mode with a brief description:

Method	Description
close()	Closes an opened file. It has no effect if the file is already closed.
detach()	Separates the underlying binary buffer from the `TextIOBase` and returns it.
fileno()	Returns an integer number (file descriptor) of the file.
flush()	Flushes the write buffer of the file stream.
isatty()	Returns `True` if the file stream is interactive.
read(`n`)	Reads at most `n` characters from the file. Reads till end of file if it is negative or `None`.
readable()	Returns `True` if the file stream can be read from.
readline(`n` =-1)	Reads and returns one line from the file. Reads in at most `n` bytes if specified.
readlines(`n` =-1)	Reads and returns a list of lines from the file. Reads in at most `n` bytes/characters if specified.
seek(`offset` , `from` = `SEEK_SET`)	Changes the file position to `offset` bytes, in reference to `from` (start, current, end).
seekable()	Returns `True` if the file stream supports random access.
tell()	Returns the current file location.
truncate(`size` = `None`)	Resizes the file stream to `size` bytes. If `size` is not specified, resizes to current location.
writable()	Returns `True` if the file stream can be written to.
write(`s`)	Writes the string `s` to the file and returns the number of characters written.
writelines(`lines`)	Writes a list of `lines` to the file.

Reading Binary File

Use the 'rb' mode in the `open()` function to read a binary files, as shown below

```
f = open(cat.png', 'rb') # opening a binary file
content = f.read() # reading all lines
content
f.close() # closing file object
```

```
>>> f=open('cat.png','rb')
>>> content=f.read()
>>> content
b'\x89PNG\r\n\x1a\n\x00\x00\x00\rIHDR\x00\x00\x08\xb8\x00\x00\x07\x80\x08\x06\x00\x00\x00\xc7~
|\xcc\x00\x00\x00\x19tEXtSoftware\x00Adobe ImageReadyq\xc9e<\x00\x00\x03diTXtXML:com.adobe.xmp
\x00\x00\x00\x00\x00<?xpacket begin="\xef\xbb\xbf" id="W5M0MpCehiHzreSzNTczkc9d"?> <x:xmpmeta
xmlns:x="adobe:ns:meta/" x:xmptk="Adobe XMP Core 5.0-c060 61.134777, 2010/02/12-17:32:00
 "> <rdf:RDF xmlns:rdf="http://www.w3.org/1999/02/22-rdf-syntax-ns#"> <rdf:Description rdf:abo
ut="" xmlns:xmpMM="http://ns.adobe.com/xap/1.0/mm/" xmlns:stRef="http://ns.adobe.com/xap/1.0/s
Type/ResourceRef#" xmlns:xmp="http://ns.adobe.com/xap/1.0/" xmpMM:OriginalDocumentID="xmp.did:
C62A2F107DDFE511895C85DBD578CF6A" xmpMM:DocumentID="xmp.did:3263775FDF7D11E5A9E9F77C9EAC0641"
xmpMM:InstanceID="xmp.iid:3263775EDF7D11E5A9E9F77C9EAC0641" xmp:CreatorTool="Adobe Photoshop C
S5 Windows"> <xmpMM:DerivedFrom stRef:instanceID="xmp.iid:C72A2F107DDFE511895C85DBD578CF6A" st
Ref:documentID="xmp.did:C62A2F107DDFE511895C85DBD578CF6A"/> </rdf:Description> </rdf:RDF> </x:
xmpmeta> <?xpacket end="r"?>\x00\xf2\x9b\xbe\x00?\xd2\x8aIDATx\xda\xec\xdd{\xac-\xd7}\x1f\xf65
\xb3\xdf\xfb\x9cs_$E\xbd\xacc)rL&Q\\\x85\xb1+\x9b\xbej\xd3 e\x8c&ma$n \xc7I\xdb\x00\x976\x1a82
\x10\xff!\x05h\x93?b\x03m\x81\xc4E\xd1\x847HQ\xc1\xb0\xd08\xb5R \x06Z\xa6mR\x90\xa6\xad8U\x1d4
I%7\x89\x9cc\x8a\xa4H\xf1>\xcec\xbf\xf7\xcct\xfd\xe6\x9c}yy\xf9\x90\x14\xd1\xbe\x9b\xbc\x9f\x0
f<\xdc{f\xd6Z\xb3f\xf6\xb9\xdcG\xbe_\xfeV\xd14M\x02\x00\x00\x00\x00\x00\x00\x80mUz\x04\x00
\x00\x00\x00\x00\x00\x00\x0013\x01\x17\x00\x00\x00\x00\x00\x00\x00\xb6\x9a\x80\x0b\x00\x00
\x00\x00\x00\x00\x00\x00[M\xc0\x05\x00\x00\x00\x00\x00\x00\x80\xad&\xe0\x02\x00\x00\x00\x0
0\x00\x00\x00\xc0V\x13p\x01\x00\x00\x00\x00\x00\x00`\xab\t\xb8\x00\x00\x00\x00\x00\x00\x00
\x00\xb0\xd5\x04\\\x00\x00\x00\x00\x00\x00\x00\xd8j\x02.\x00\x00\x00\x00\x00\x00\x00\x0015
\x01\x17\x00\x00\x00\x00\x00\x00\x00\xb6\x9a\x80\x0b\x00\x00\x00\x00\x00\x00\x00\x00[M\xc0
\x05\x00\x00\x00\x00\x00\x00\x80\xad&\xe0\x02\x00\x00\x00\x00\x00\x00\xc0V\x13p\x01\x0
0\x00\x00\x00\x00\x00`\xab\t\xb8\x00\x00\x00\x00\x00\x00\x00\xb0\xd5\x04\\\x00\x00\x00
\x00\x00\x00\x00\x00\xd8j\x02.\x00\x00\x00\x00\x00\x00\x00\x0015\x01\x17\x00\x00\x00\x00\x00
00\x00\x00\xb6\x9a\x80\x0b\x00\x00\x00\x00\x00\x00\x00[M\xc0\x05\x00\x00\x00\x00\x00\x00\x
```

Writing to a Binary File

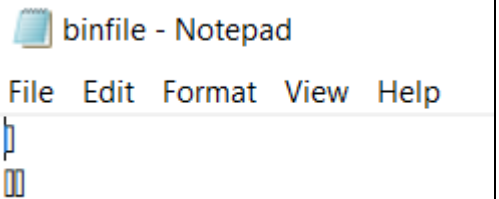
The **open()** function opens a file in text format by default. To open a file in binary format, add **'b'** to the mode parameter. Hence the **"rb"** mode opens the file in binary format for reading, while the **"wb"** mode opens the file in binary format for writing. Unlike text files, binary files are not human-readable. When opened using any text editor, the data is unrecognizable.

The following code stores a list of numbers in a binary file. The list is first converted in a byte array before writing. The built-in function bytearray() returns a byte representation of the object.

.bin file indicates binary file

```
>>> f=open("binfile.bin","wb")
>>> num=[5,10,15,30,24]
>>> arr=bytearray(num)
>>> f.write(arr)
5
>>> f.close()
>>>
```

```
f=open("binfile.bin","wb")
num=[5, 10, 15, 20, 25]
arr=bytearray(num)
f.write(arr)
f.close()
```



Question Bank

[1]What are the two types of files? Explain different file operations
[2]Define File.
[3] Write a Python program to demonstrate the file I/O operations
[4] Discuss with suitable examples
i) Close a File.
ii) Writing to a File.
[5]List the file opening Modes.
[6] List the different ways to read a file.
[7] What the difference is between append and write mode?
[8] What are the attributes of file objects?
[9] List the methods in file objects.
[10]Point out different modes of file opening
[11]Define the access modes
[12]Define read and write file.
[13]Describe renaming and delete.
[14]Write a program to write a data in a file for both write and append modes.
[15]Explain with example of closing a file
[16]Discover syntax for reading from a file
[17]Structure Renaming a file
[18]Explain about the Files Related Methods.
[19]How to open a new file in Python?
[20]Explain how the write method works on a file.
[21]Which method is used to read the contents of a file which is already created?
[22]What is a text file? Give an example for a text file.